

Actividad 2: Creación de un prototipo en Unity

Máster: Máster Universitario en Diseño de Videojuegos

Asignatura: Motores y Herramientas de Desarrollo

Integrantes del grupo:

- Alejandro Quijada Leyton
- Axel Castorena Rivera
- Maria Vilaseca Nadal
- Valentina Liberona Zúñiga

Repositorio del Proyecto:

<https://gitlab.com/valeLib/uem2g4a2/-/tree/3f4822bba665deef4fb30629e77628026a3e73b7/>

1. Assets utilizados

- **2D Platformer**

El **2D Platformer Game Toolkit** es un paquete gratuito que proporciona una base completa para crear juegos de plataformas 2D en Unity, incluyendo un personaje jugable totalmente funcional (movimiento, salto, ataque, animaciones), enemigos básicos, pickups, colisiones, físicas configuradas y escenas de ejemplo. Sirve como *starter kit* para prototipar o desarrollar rápidamente un platformer 2D sin tener que implementar desde cero la lógica principal del personaje, la estructura del nivel o las interacciones básicas del juego.

url:

<https://assetstore.unity.com/packages/tools/game-toolkits/2d-platformer-229878>

- **Lean GUI**

Lean GUI es un **asset gratuito** que amplía la UI de Unity añadiendo componentes listos para usar como botones mejorados, animaciones de UI, tooltips, ventanas, sliders, entre otros. Sirve para crear interfaces más interactivas, fluidas y modernas sin tener que programar toda la lógica desde cero.

url: <https://assetstore.unity.com/packages/tools/gui/lean-gui-72138>

● Tiles para nuevos Assets

El **Platform Tile Pack** es un paquete gratuito que incluye una serie de tiles en 2D, el cual fue importado al proyecto para añadir sprites a los assets necesarios para nuevas mecánicas.

url:

<https://assetstore.unity.com/packages/2d/environments/platform-tile-pack-204101>

● 8Bit Music - 062022

El **8Bit Music** es un asset gratuito que contiene 10 tracks con temas **8bit**, **Retro** y **Electronic**.

url:

<https://assetstore.unity.com/packages/audio/music/8bit-music-062022-225623>

2. Mecánicas Jugables Añadidas

2.1 Mecánica: Stomp al Enemigo

Script modificado: *PlayerController.cs*

Descripción del funcionamiento

1. Validación del enemigo

- Se comprueba si el objeto colisionado posee el componente **EnemyAI**.
- Solo los enemigos con este componente pueden ser pisados; obstáculos como pinchos no activan esta mecánica.

2. Análisis del contacto

- Se obtiene la normal del punto de colisión, lo que permite saber desde qué dirección ocurrió el impacto.

3. Verificación de stomp válido

- Gravedad normal: el stomp es válido si **contactNormal.y** > **0.5f** (el jugador golpea desde arriba).
- Gravedad invertida: el stomp es válido si **contactNormal.y** < **-0.5f** (impacto desde abajo).

4. Ejecución del stomp

Cuando el stomp es correcto:

1. Se llama a **enemy.Die()** para eliminar al enemigo.
2. Se reproduce el sonido de muerte (si existe **deathSFX** asignado) mediante **audioSource.PlayOneShot()**.
3. Se aplica un rebote al jugador usando una fuerza equivalente a **jumpForce** * **0.5f**, en dirección opuesta a la gravedad.
4. Se notifica al controlador global mediante **GameStatusController.instance.EnemyDefeated(gameObject)**, lo que actualiza contadores y condiciones de victoria.
5. Se finaliza la ejecución de la función para evitar activar la lógica de muerte del jugador.

5. Eliminación del enemigo

- Se calcula un retraso basado en la duración del sonido de muerte (**deathSFX.length**).
- Si no hay sonido, se usa un tiempo mínimo de 0.1 segundos.
- El **GameObject** del enemigo se destruye después del retraso, permitiendo que el sonido se reproduzca completamente antes de eliminar el **AudioSource**.

2.2 Mecánica: Mecánica de Inversión de gravedad

Script Modificado: *GreenStone.cs* y *SoundEffect.cs*.

Descripción del funcionamiento

1. Detección del jugador:

- Al producirse una colisión con el trigger del objeto, se verifica si el collider que ingresó posee la etiqueta "Player".
- Se inicia el proceso de desactivación temporal del objeto.

2. Reproducción de sonidos:

- Se ejecuta la reproducción inmediatamente tras la colisión mediante: **audioSource.PlayOneShot(pickupSound)**.
- Una vez completado el tiempo de inversión de gravedad, se verifica si existe un **effectEndSound**. Se reproduce con **audioSource.PlayOneShot(effectEndSound)**.

3. Inversión de gravedad:

- El jugador ejecuta una corrutina interna mediante: **player.StartCoroutine(player.InvertGravity(gravityTime))**.
- Esta corrutina invierte la gravedad del jugador por un tiempo definido en **gravityTime**.

4. Finalización del efecto:

- Cuando la corrutina del jugador termina, se reproduce el sonido final.
- No se aplican efectos adicionales hasta que la piedra reaparezca.

5. Respawn de GreenStone:

Desactivación inmediata al activarse

- El SpriteRenderer se deshabilita para ocultar visualmente el objeto.
- El Collider2D se desactiva para impedir múltiples activaciones consecutivas.

Tiempo de reaparición

- El objeto permanece desactivado por un valor configurado en **respawnTime**.

Reactivación

- Pasado este tiempo:
 - El sprite vuelve a habilitarse.
 - El collider se reactiva.
- La piedra queda nuevamente disponible para el jugador.

3. Condición de Victoria y/o derrota

GameStatusController.cs maneja el fin de la partida usando un **singleton**.

Al iniciar, el script utiliza un arreglo serializado de enemigos (**enemies[]**) configurado desde el Inspector de Unity, y crea un arreglo **enemiesAlive[]** para registrar cuáles siguen vivos.

Cuando un enemigo muere, el método **EnemyDefeated()** marca a ese enemigo como derrotado y luego un **coroutine** verifica si todos han sido eliminados; si es así, se activa la victoria y se carga el siguiente nivel o escena tras 3 segundos.

Para la derrota, cuando el jugador muere se llama **PlayerDied()**, lo cual activa la derrota inmediatamente y carga la escena de Game Over.

El controlador usa un flag **gameEnded** para evitar múltiples activaciones y un pequeño retraso en la verificación de victoria para evitar conflictos al destruir enemigos. Ahora incluye referencias a escenas mediante **SceneReference** para el siguiente nivel y la escena de Game Over, y utiliza el LevelManager para las transiciones de escena.

Scripts Modificados

Gestión de Victoria

1. Inicialización:

- Utiliza un arreglo serializado **enemies[]** configurado desde el Inspector (no busca enemigos por etiqueta).
- Crea un arreglo booleano **enemiesAlive[]** para registrar el estado de cada enemigo.
- Inicializa todos los enemigos como vivos (true).

2. Registro de derrota de enemigos:

- Verifica **gameEnded** para evitar procesamiento si el juego ya terminó.
- Se llama cuando un enemigo muere (probablemente desde el script del propio enemigo).
- Busca en el arreglo **enemies[]** para identificar cuál enemigo fue derrotado.
- Marca la entrada correspondiente **enemiesAlive[i]** como **false**.

- Activa una comprobación de victoria mediante una **coroutine**.
3. **Comprobación de victoria:**
 - Espera un frame para asegurar que el enemigo haya sido completamente destruido.
 - Llama a **CheckAllEnemiesDefeated()**, que recorre el arreglo **enemiesAlive[]**.
 - Si todas las entradas son **false** (todos derrotados), ejecuta **Victory()**.
 4. **Verificación de enemigos:**
 - Recorre el arreglo **enemiesAlive[]** buscando enemigos vivos.
 - Retorna false si encuentra al menos un enemigo vivo.
 - Retorna true solo si todos están derrotados.
 5. **Estado de victoria:**
 - Establece **gameEnded = true** para evitar más cambios de estado.
 - Registra un mensaje de victoria.
 - Carga el siguiente nivel después de 3 segundos usando **Invoke()**.
 6. **Carga del siguiente nivel:**
 - Si **nextLevelScene** está asignado, usa **LevelManager.Instance.LoadScene()**.

Gestión de Derrota

1. **Muerte del jugador:**
 - Se llama cuando el jugador muere (probablemente desde **PlayerMovement** u otro script relacionado).
 - Activa directamente **Defeat()**.
2. **Estado de derrota:**
 - Establece **gameEnded = true** para evitar más cambios de estado.
 - Registra un mensaje de derrota.

4. Menús e interfaces añadidas

Para gestionar la navegación del jugador se agregaron las siguientes escenas dedicadas a UI:

- **MainMenu:** menú principal del juego.
- **Level_1** y **Level_2:** niveles jugables, que incluyen el menú de pausa superpuesto.
- **Victory:** pantalla de victoria al completar un nivel.

- **GameOver:** pantalla de derrota.

De esta forma la interfaz guía al jugador desde la entrada al juego hasta el final de la partida, incluyendo estados intermedios como pausa, victoria y derrota.

Menú Principal: Escena *MainMenu*

En la escena **MainMenu** se implementó el **menú principal** del juego:



- **Elementos interactivos**
 - Botón **JUGAR**: el juego abandona la escena de Intro y carga el primer nivel jugable (Level_1).
 - Botón **SALIR**: la aplicación se cierra.

Este menú cumple la función de punto de entrada, permitiendo al jugador iniciar una partida o salir del juego de forma clara y directa.

Pantalla de Victoria: Escena *Victory*

La escena **Victory** muestra la **pantalla de victoria** que aparece cuando el jugador supera el nivel:



- **Elementos interactivos**

- Botón **Jugar Otra Vez**: permite reiniciar la experiencia jugable (recargando el último nivel o el Level_1).
- Botón **Volver al Menú Principal**: regresa a la escena **MainMenu**, desde donde se puede iniciar una nueva partida o salir.

Esta pantalla cierra el ciclo de juego cuando el jugador gana e introduce opciones claras para seguir jugando o volver al menú.

Menú de Pausa: Escenas *Level_1* y *Level_2*

En las escenas jugables se añadió un **menú de pausa** superpuesto a la acción:



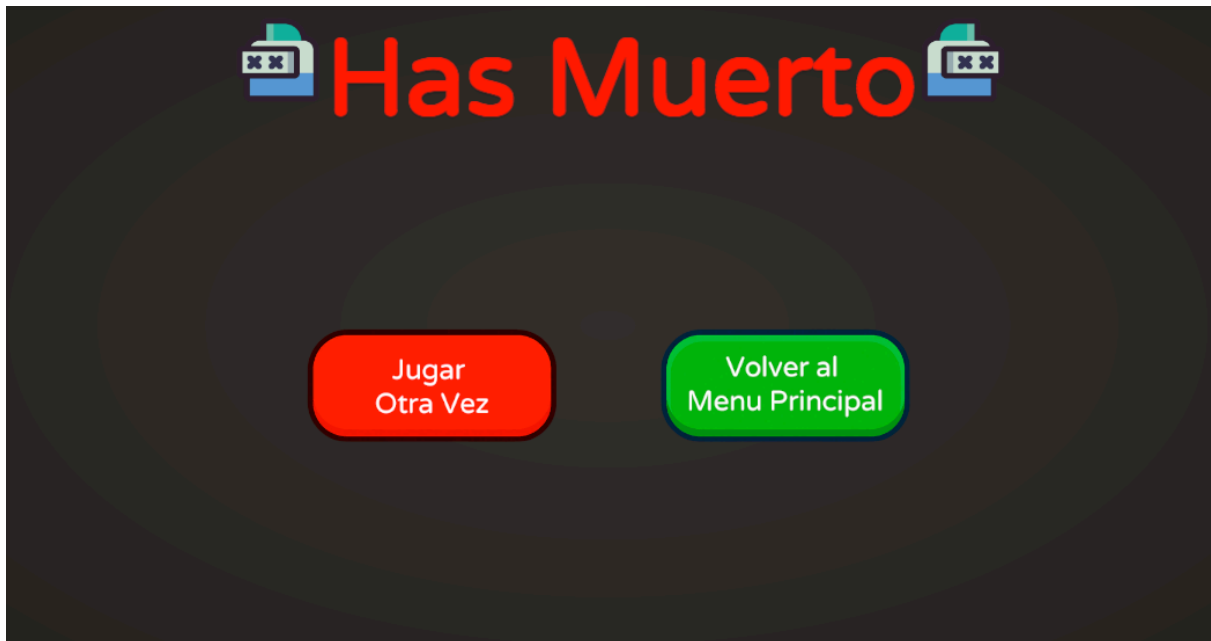
- **Comportamiento**

- El menú se activa cuando el jugador presiona la tecla **P** o **Esc** durante el juego.
- Al activarse:
 - Se muestra el panel de pausa sobre la escena sin cambiar de nivel.
 - El juego entra en estado pausado (mediante ***Time.timeScale = 0***), deteniendo la lógica jugable.
- Al volver a presionar **P** o **Esc**, el panel se oculta y el juego se reanuda.

Gracias a este sistema, el jugador puede detener la partida en cualquier momento para descansar sin perder el progreso actual.

Pantalla de Derrota: Escena *GameOver*

La escena **GameOver** corresponde a la **pantalla de derrota**, mostrada cuando el jugador pierde la partida (por ejemplo, al caer en una trampa o ser derrotado por un enemigo).



- **Elementos interactivos**

- Botón **Jugar Otra Vez**: reinicia el nivel actual, permitiendo al jugador volver a intentarlo inmediatamente.
- Botón **Volver al Menú Principal**: regresa a la escena **MainMenu**, cerrando el ciclo de juego y permitiendo iniciar una nueva partida desde cero.

Esta pantalla cumple la función de informar al jugador sobre el estado de derrota, ofrecer opciones claras para continuar o abandonar, y mantener consistencia estética con las demás interfaces del proyecto.

5. Descripción de las Escenas

Escenas de Interfaz

El proyecto incluye varias escenas dedicadas exclusivamente a la navegación del juego:

- **MainMenu**: menú principal con botones para Jugar y Salir.
- **Victory**: pantalla de victoria tras completar un nivel, con opciones para reiniciar o volver al menú.
- **GameOver**: pantalla de derrota, con opciones para reiniciar o volver al menú.

Estas escenas y paneles cumplen la función de guiar al jugador entre los estados principales del juego, pero no contienen lógica jugable. Su objetivo es entregar una experiencia de navegación clara y coherente.

Escena Level_1

La escena **Level_1** corresponde al primer nivel jugable del proyecto. Incluye todos los elementos necesarios para la experiencia de plataforma:

Estructura general

- Terreno principal diseñado con plataformas a diferentes alturas.
- Varias zonas de peligro marcadas con **spikes** (pinchos).
- Distribución balanceada de **enemigos**, **monedas** y **coleccionables verdes** para motivar exploración y movimiento.

Objetos principales

- **Player:** el personaje controlado por el usuario, con scripts de movimiento, salto, detección de colisiones y mecánicas especiales como stomp.
- **Enemies:** enemigos básicos con comportamiento simple de patrulla o presencia estática.
- **Coins:** coleccionables que incrementan la puntuación.
- **Platforms:** plataformas flotantes que permiten navegar verticalmente.
- **Items:** caja interactuable y otros objetos que forman parte del diseño del nivel.

Sistemas incluidos

- **GameManager / GameStatus:** controlan el estado de la partida (victoria/derrota) y verifican condiciones como eliminación de enemigos.
- **Pause UI:** panel de pausa accesible durante el juego.
- **EventSystem + Canvas:** soporte para UI dentro de la escena.

Diseño del recorrido

El layout del nivel obliga al jugador a moverse horizontal y verticalmente, saltar entre plataformas, evitar trampas y enfrentar enemigos. Al llegar al final del nivel y cumplir las condiciones de victoria, se carga la escena Victory.

Escena Level_2

La escena **Level_2** replica la estructura básica del **Level_1**, pero con variaciones en su diseño para aumentar dificultad o presentar un desafío distinto:

Modificaciones clave

- Pinchos que cubren todo el suelo y el techo para aumentar la dificultad.
- Plataformas con mayor separación vertical y más estrechas para exigir saltos más precisos.
- Nuevas combinaciones de trampas y coleccionables.
- Diferentes rutas para la libertad del jugador.
- El objetivo sigue siendo alcanzar el final del nivel para activar la pantalla de Victoria.
- Cambio de colores del escenario para diferenciarlo del **Level_1**.

Level_2 demuestra la capacidad del proyecto para escalar la dificultad reutilizando sistemas ya implementados (UI, GameManager, player controller) y variando únicamente el diseño del entorno y la ubicación de los elementos.

6. Prefabs reutilizables

Prefab: WoodenBox

WoodenBox es un prefab creado a partir del paquete de tiles incluido en el proyecto.

Representa una caja de madera con la que tanto el jugador como los enemigos pueden interactuar. Su propósito principal es servir como objeto físico empujable, permitiendo alcanzar plataformas más altas o bloquear el avance de enemigos.

Para cumplir estas funciones, el prefab incorpora los componentes **Rigidbody2D** y **BoxCollider2D**, que habilitan su comportamiento físico dentro del entorno 2D. También incluye un **AudioSource** que reproduce un sonido cada vez que el jugador genera una interacción relevante (como empujarla o colisionar). Además, posee un script asociado que gestiona esta interacción y la reproducción del audio.

Este prefab es utilizado tanto en el **Nivel 1** como en el **Nivel 2**.

Prefab: GreenStone

GreenStone es un prefab creado a partir del mismo paquete de tiles que *WoodenBox*, aunque su función es completamente distinta. Se trata de una piedra especial que, al ser tocada por el personaje principal, **activa un cambio temporal de gravedad**, modificando momentáneamente la jugabilidad y el desplazamiento del personaje.

El prefab cuenta con un **AudioSource** que reproduce un efecto sonoro al activarse, y con los scripts **GreenStone** y **GreenStoneSound**, encargados de gestionar tanto la inversión de la gravedad como la reproducción del audio asociado al evento.

Este prefab es utilizado tanto en el **Nivel 1** como en el **Nivel 2**.

7. Instanciación de objetos

El script **PrefabSpawner.cs** es un componente añadido al proyecto que, al ser asignado a un **GameObject** dentro de una escena de **Unity**, se encarga de instanciar múltiples **prefabs** en posiciones específicas cuando la escena comienza.

El script está compuesto por:

- **Campos serializados:**
 - **prefabs:** Arreglo de *GameObjects* que contiene los prefabs a instanciar.
 - **spawnPositions:** Arreglo de *Vector2* que define la posición donde se instanciará cada prefab.
- **Ejecucion: En Awake() se realiza el siguiente proceso:**
 1. Validación de campos:
 - Verifica que los arreglos **prefabs** y **spawnPositions** tengan la misma cantidad de elementos.
 - Si los tamaños no coinciden, se muestra un mensaje de error en consola y se detiene la ejecución para evitar instancias desalineadas.
 2. Bucle de instanciación:
 - Recorre el arreglo de **prefabs**.
 - Comprueba que cada referencia no sea null.
 - Instancia el prefab correspondiente en **spawnPositions[i]**.

8. Sonidos y música

Sonidos

- Sonido asignado a la interacción con el prefab GreenStone, audio llamado 'jump' en el proyecto: https://freesound.org/people/Xupr_e3/sounds/686523/
- Sonido asignado a la interacción con el prefab Coin, audio llamado 'coin' en el proyecto:
<https://freesound.org/people/LilMati/sounds/402767/>
- Sonido asignado a la interacción con el prefab WoodenBox, audio llamado 'wood' en el proyecto:
<https://freesound.org/people/Krokulator/sounds/652615/>
- Sonido asignado a la finalización del efecto de GreenStone, audio llamado 'down' en el proyecto:
<https://freesound.org/people/astrand/sounds/328011/>
- Sonido asignado a la muerte del prefab Enemy, audio llamado 'ghost' en el proyecto:
<https://freesound.org/people/LittleRobotSoundFactory/sounds/316414/>

Musica de fondo

Intro: Track 2

Se ha elegido esta canción por su ritmo alegre y enérgico. Encaja con la atmósfera de la pantalla de introducción (colores vibrantes, el personaje bailando), y también anima al jugador.

Level_1: Track 9

Esta canción sigue siendo animada, pero un poco más oscura para indicar que habrá peligro que el jugador tendrá que afrontar. Sin embargo, es el primer nivel y bastante simple, así que el peligro no es tan grave. Por lo tanto, el ritmo animado es apropiado.

Level_2: Track 4

La Track 4 es más seria para indicar que el nivel 2 es más peligroso que el anterior.

Victory: Track 5

La música es alegre y relajada para indicar que el éxito y que ya no hay más enemigos con los que lidiar.

Game Over: Track 10

Esta música es más sombría para indicar la derrota, pero se vuelve más energética para incentivar al jugador a intentarlo de nuevo.

9. Elementos persistentes

Se implementó un sistema para manejar la navegación entre niveles y pantallas de UI utilizando **objetos persistentes**, principalmente a través del script **LevelManager**, y complementándose con **MainMenu** y **GameStatusController**.

Este enfoque asegura que las funciones de carga de escenas estén centralizadas, sean reutilizables y no se dupliquen en cada nivel.

LevelManager (Persistente entre escenas)

LevelManager es el componente central del sistema de navegación. Su responsabilidad es cargar niveles, reiniciar la escena actual y cerrar el juego.

Razón de ser un elemento persistente

El script implementa un **Singleton persistente**:

- Crea una única instancia accesible globalmente.
- Usa ***DontDestroyOnLoad(gameObject)*** para mantenerse entre cambios de escena.
- Si otra escena intenta crear uno nuevo, se destruye automáticamente.

Esto permite que cualquier parte del proyecto pueda cargar niveles sin depender de objetos específicos de una escena en particular.

Funciones principales

- **LoadScene(SceneReference):** carga una escena utilizando un SceneReference.
- **ReloadCurrentScene():** reinicia el nivel actual.
- **QuitGame():** cierra la aplicación (o detiene el editor en modo Play).

Beneficio: todo el manejo de escenas queda centralizado y estandarizado.

MainMenu (Interfaz conectada al LevelManager)

El script **MainMenu** controla los botones del menú principal. No es persistente, pero se apoya en **LevelManager**, que sí lo es.

Funciones clave

- **PlayGame():** llama a *LevelManager.Instance.LoadScene()* para cargar el primer nivel.
- **QuitGame():** usa el mismo LevelManager para salir del juego.

Relevancia en el sistema persistente

El menú no necesita lógica de navegación propia; delega todo en el **objeto persistente LevelManager**, reduciendo duplicación y errores.

GameStatusController (Coordinación con el sistema persistente)

Este script controla el estado de victoria o derrota en cada nivel. Aunque **no es persistente**, usa referencias de escenas y delega la carga en LevelManager.

Integración con persistencia

GameStatusController no necesita implementar carga de escenas:

solamente informa el estado (victoria/derrota), y **LevelManager** se encarga del resto.

Esto separa responsabilidades:

- **GameStatusController:** reglas del juego.
- **LevelManager:** transición global de escenas.

GameOverMenu (Interfaz final que reutiliza el sistema persistente)

El script **GameOverMenu** controla la pantalla de derrota y permite reiniciar el nivel o volver al menú principal.

Ambos botones llaman a **LevelManager.Instance**, reforzando la idea del LevelManager como único controlador de navegación.

GameManage (Uso indirecto del sistema persistente)

GameManager maneja el conteo de monedas y la animación de muerte. Cuando detecta la muerte del jugador, reinicia la escena: *LevelManager.Instance.ReloadCurrentScene()*;

Aunque no es un elemento persistente, **depende del objeto persistente para reiniciar el nivel**, manteniendo coherencia con el resto del proyecto.

10. Scripting Avanzado

GameStatusManager

En **GameStatusController.cs** se aplica programación avanzada mediante el uso de arreglos paralelos (**enemies[]** y **enemiesAlive[]**) que mantienen sincronizadas las referencias de enemigos con su estado, permitiendo un seguimiento eficiente sin consultar GameObjects en tiempo real. La corrutina **CheckVictoryCoroutine()** introduce programación asíncrona con `yield return null` para diferir la verificación de victoria un frame y evitar problemas de sincronización cuando Unity aún no ha completado la destrucción de objetos. Además, el patrón Singleton asegura una única instancia global del controlador, y el uso de `break` y `return` optimiza los bucles finalizándolos tan pronto como se obtiene el resultado necesario. En conjunto, esta arquitectura demuestra manejo avanzado de flujo asíncrono, gestión eficiente de colecciones y comprensión del ciclo de vida interno de Unity.

1. Corrutina **CheckVictoryCoroutine()**

- Espera un frame antes de verificar si todos los enemigos han sido derrotados.

- Permite que Unity complete la destrucción del enemigo antes de verificar el estado de victoria, evitando condiciones de carrera.
- Se inicia en la **línea 62** con **StartCoroutine(CheckVictoryCoroutine())**.
- Elemento clave: **yield return null** pausa la ejecución por un frame.

2. Arreglos:

- **Línea 17:** Declaración del arreglo de enemigos [**SerializeField**] **private GameObject[] enemies**. Arreglo de **GameObjects** que contiene referencias a todos los enemigos (**configurado desde el Inspector**)
- **Línea 18:** Declaración del arreglo de estado **private bool[] enemiesAlive**. Arreglo de booleanos que rastrea qué enemigos están vivos (**true**) o muertos (**false**).
- **Línea 37:** Inicialización del arreglo booleano **enemiesAlive = new bool[enemies.Length]**. El tamaño de **enemiesAlive[]** se ajusta automáticamente al número de enemigos en **enemies[]**.

3. Bucles:

- **Líneas 40-43:** Inicialización de enemigos vivos, marca todos los enemigos como vivos al inicio del juego y recorre todo el arreglo **enemiesAlive[]**.
- **Líneas 52-59:** Búsqueda del enemigo derrotado. Encuentra qué enemigo específico fue derrotado y lo marca como muerto y recorre el arreglo **enemies[]** hasta encontrar coincidencia. Usa **break** para salir del bucle inmediatamente después de encontrar el enemigo
- **Líneas 82-89:** Verificación de victoria. Verifica si quedan enemigos vivos. Recorre todo el arreglo **enemiesAlive[]** y si encuentra aunque sea un enemigo vivo (**true**), retorna **false** inmediatamente.

11. Exportación del Proyecto

La versión ejecutable del juego ha sido generada para **Windows** y se encuentra disponible en el archivo:

- **ActividadCreacionPrototipo_build.zip**

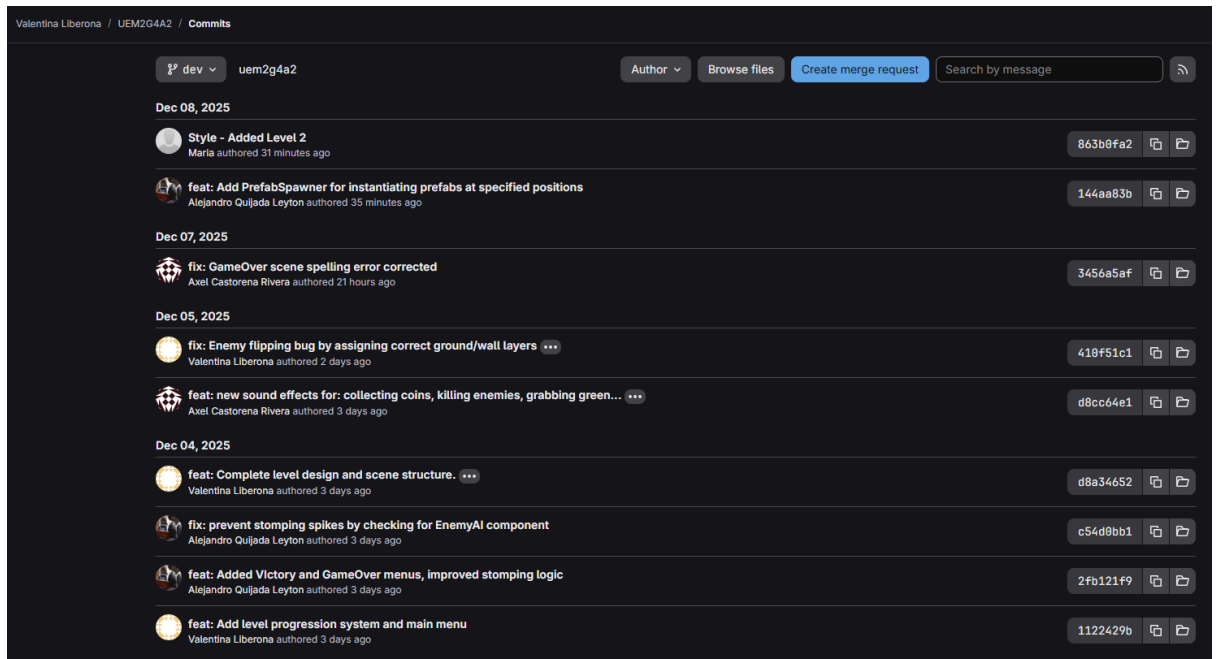
Asimismo, el proyecto completo de Unity, incluyendo todos los assets, escenas y scripts utilizados, está disponible en:

- **ActividadCreacionPrototipo_Unity.zip**

12. Git

Capturas del repositorio

- Historial de Commits:



- Manejo de branches: *main* que es protegido, y *dev* donde se realizó todo el desarrollo del proyecto.

Valentina Liberona / UEM2G4A2

U **UEM2G4A2** 🔒

main uem2g4a2 + Find file Code

Select Git revision

Search by Git revision

Selected

- main (default) protected

Branches 1

- dev
- ProjectSettings

	Last commit	Last update
	fix: Enemy flipping bug by assigning co...	1 minute ago
	Initial commit	4 days ago
	fix: Enemy flipping bug by assigning co...	1 minute ago
.gitignore	feat: Add level progression system and...	23 hours ago
.vsconfig	feat: Added tiles package. New anti gra...	1 day ago
README.md	Initial commit	4 days ago
UEM2G4A2.slnx	chore: added SceneReference helper	2 days ago

- Merge request desde *branch dev* a *branch main*.

Valentina Liberona / UEM2G4A2 / Merge requests / 11

feat: Added tiles package. New anti gravity mechanich with green stone

Merged Valentina Liberona requested to merge dev into main just now

Overview 0 Commits 0 Pipelines 0 Changes 0

feat: Added tiles package. New anti gravity mechanich with green stone. New prefab: Wooden box. New prefab: Ladder

👍 0 🗨️ 0 😊

8 Approval is optional

Merged by Valentina Liberona 1 minute ago

Revert Cherry-pick Delete source branch

Merge details

- Changes merged into main with [a72d8582](#).
- Did not delete the source branch.